SmartRail – UX Process Method

Recommended approach for the requirement discovery phase

Recommended approach: Double Diamond, with Lean UX loops inside it

Use **Double Diamond as the overall frame** for the project. Then use **Lean UX's Think → Make → Check loop** inside the second diamond, when you design and build. The reasons come straight from the assessment brief.

Why Double Diamond fits

1. The brief separates problem-finding from problem-solving

Section 22 says discovering what is not specified is assessed in its own right. Section 23 then asks for requirements artifacts before any design or build artifacts. Double Diamond keeps those two phases visibly apart, so assessors can see your discovery work on its own.

2. Section 25 follows the same diverge-then-converge shape

The brief asks you to "explore the broader problem → identify important capabilities → define achievable scope → prioritise → MVP → document deferrals." That sequence is the first diamond:

- **Discover** explores the whole SmartRail ecosystem.
- **Define** narrows it to a scoped, prioritised MVP.

The deferred-features list falls out of the Define phase naturally.

3. It gives you a clear story for the final demo

Section 27 wants a narrative rather than a screen tour. A natural structure is: "Here's what I discovered, here's how I narrowed it, here's what I built, here's how I validated it."

4. It works for a multi-app system

You have five applications serving very different users. Double Diamond lets you discover across all actors first, then make one consistent set of scope decisions. Without that, each app tends to drift into its own mini-project.

Why not the others as the main method

- **Design Thinking (IDEO)** is close, but it is research-heavy and assumes access to real users. You will mostly be simulating stakeholders with AI, so a method built around deep empathy research would overstate what you can actually do.
- **Lean UX alone** starts from assumptions and skips structured discovery. That is exactly the part the brief grades most heavily.
- **Agile UX** assumes a team where design runs a sprint ahead of development. You are one person, so that hand-off rhythm doesn't really exist.

Where Lean UX comes in

Once you reach **Develop/Deliver**, switch to short Think → Make → Check loops. For example:

1. Wireframe the "where is my train now" flow.

2. Have an AI agent play a rushed commuter and critique it.
3. Revise, then build.

This also handles an honest limitation in your process. AI-simulated stakeholders are not real users. Lean UX lets you state their input explicitly as **assumptions to validate**, rather than presenting it as research findings. Assessors will likely value that candour under section 21.

How the phases map to your deliverables

Phase	What you do	Section 23 artifacts produced
Discover	AI stakeholder simulations (passenger, driver, controller, catering provider, security, accessibility…), gap analysis of the brief	Raw findings, missing actors/states/failure cases, conflicts, ambiguities
Define	Consolidate, prioritise (e.g. MoSCoW), scope the MVP	Refined requirements, assumptions log, personas, user journeys, user stories plus acceptance criteria, edge cases, deferred list
Develop	Lean UX loops on flows and designs, architecture exploration	User flows, wireframes, design system, architecture and ADRs
Deliver	Build, test, validate against acceptance criteria	Working apps, tests, defects, quality assessment

AI-usage evidence to capture during Discover

Section 21 asks you to show how you validated AI output. While you run the stakeholder simulations, log three things:

- **The prompts you used**, for example: “You are a station manager at a busy interchange; what goes wrong on a platform change?”
- **What you accepted.**
- **What you rejected, and why.** For example, the AI might invent a ticketing or payment feature that is outside scope, or ignore that the catering provider must stay a separate system.

Next step

Start the Discover phase: run a structured simulation with each stakeholder role and turn the output into a first gap list of missing actors, failure scenarios, states, and conflicts.